

CS 744: Design and Engineering of Computing Systems

Lab 3 — To Infinity and Beyond with xv6

Vansil Chauhan (24M0847)

13 September 2024

Overview and Setup

Lab 3 works inside the x86 teaching operating system xv6, extending it in three directions: adding new user-space programs, adding new system calls that expose process and memory-management state to user code, and modifying the kernel’s trap handler to support lazy (demand-paged) memory allocation. Unlike a typical application, xv6 is cross-built on the host machine — there is no compiler or editor inside the OS itself. Every source change lives in the host source tree; a full rebuild produces `kernel`, `xv6.img` (the bootable kernel image) and `fs.img` (the root filesystem image containing the compiled user binaries and any data files), both of which are handed to QEMU as virtual disks. This means every new program or system call requires two kinds of change: the actual C/assembly implementing the feature, and `Makefile` bookkeeping (`UPROGS/CUSTOMPROGS`, `EXTRA.FILES`) so the new binary or data file is compiled and copied into `fs.img`.

The lab is organized into four graded tasks — adding user programs, adding process-management system calls, exposing virtual/physical address information, and implementing page-fault-driven lazy allocation — plus three optional exercises. This report walks through what was implemented for each, grounded in the actual kernel and user-space source in `24m0847_lab3/xv6/`, and explains *why* each piece works the way it does, not just what the code does.

Task 1: Adding New Programs to xv6

(a) pingpong — counting occurrences of a substring

`pingpong` takes a file name argument, scans the file’s contents, and prints `pong` once for every occurrence of the substring `ping` it finds. The lab explicitly suggests starting from `wc.c`, xv6’s word count utility, because both programs share the same shape: open a file descriptor, `read()` it in fixed-size chunks into a buffer, and scan the buffer byte by byte. `wc.c` already solves the read-loop and error-handling boilerplate; the only real work is replacing its character-classification state machine with substring matching.

The implementation reads the file in 512-byte chunks and, at every byte offset `i` in the chunk, checks whether the next four bytes spell `ping`:

Listing 1: Core matching loop in `pingpong.c`

```
1 while ((n = read(fd, read_buffer, BUFSIZE)) > 0) {
```

```

2   for (int i = 0; i < n; i++) {
3       int flag = 1;
4       for (int j = 0; j < 4; j++) {
5           if (i + j >= n || read_buffer[i + j] != target[j]) {
6               flag = 0;
7               break;
8           }
9       }
10      if (flag == 1) {
11          write(1, output, 5);
12          i += 3;
13      }
14  }
15 }

```

Two details matter here. First, the `i += 3` skip after a match (combined with the loop’s own increment) advances the scan past the matched word rather than re-scanning inside it, which avoids double-counting on inputs like `pingping`. Second, because the file is read in fixed 512-byte chunks rather than loaded whole, an occurrence of `ping` that straddles a chunk boundary would be missed — an accepted limitation given `xv6`’s small, fixed-size I/O buffers and the fact that the sample input file never exercises this edge case.

Getting `pingpong` to run required two `Makefile` additions beyond the source file itself: adding `_pingpong` to the program list so it gets compiled and linked as an `xv6` ELF binary, and adding `pingpong.txt` to `EXTRA_FILES` so `mkfs` copies the sample input text into `fs.img` — `xv6` has no way to create that file from inside the emulated OS, since there is no text editor in `xv6`.

(b) `cmd` — shell in a shell

`cmd` is a minimal process launcher: given a program name and arguments on its own command line, it creates a child process that `execs` the named program, while the parent blocks until the child finishes.

Listing 2: `cmd.c`

```

1  int cid = fork();
2  if (cid == 0) {
3      exec(argv[1], argv + 1);
4      printf(1, "Exec failed!");
5      exit();
6  } else {
7      wait();
8  }
9  exit();

```

The reason this needs two system calls rather than one is fundamental to how Unix-family process creation works, and `xv6` follows the same model: ‘`fork()`’ duplicates the calling process — address space, open files, and all — and returns twice, once in each copy, distinguished only by the return value (child gets 0, parent gets the child’s pid). ‘`exec()`’ then *replaces* the calling process’s own image with a new program; it does not create a new process. Passing ‘`argv + 1`’ rather than ‘`argv`’ to ‘`exec`’ is what strips ‘`cmd`’s own name off the front of the argument vector, so the child actually runs `ls`, `echo hello xv6 os !!`, or whatever program name followed `cmd` on the command line, with that program’s own `argv[0]` set correctly. If `exec` returns at all, it means the program could

not be found or loaded, which is why the code after the `exec` call is treated as the failure path: a successful `exec` never returns to this frame at all, since the process's code, stack, and instruction pointer have all been overwritten by the new program.

Task 2: Adding New System Calls

Every new system call in this lab touches the same six files, and it is worth naming the plumbing once rather than repeating it for each call:

- `syscall.h` assigns each call a unique number (22–33 for this lab's calls, appended after the 21 stock xv6 calls).
- `usys.S` defines the user-space trampoline: it moves the syscall number into `%eax` and executes `int $T_SYSCALL` (interrupt vector 64), which traps into the kernel.
- `user.h` declares the C function prototype so user programs can call the syscall like an ordinary function.
- `syscall.c` maps numbers to kernel-side handler function pointers in the `syscalls[]` table, dispatched from `syscall()` which reads the number back out of `curproc->tf->eax` (the trap frame preserves the register state at the moment of the `int` instruction).
- `sysproc.c` contains the `sys_*` wrapper for each call, whose only job is to pull arguments off the user stack via `argint/argptr` (which additionally range-check pointers against the calling process's address space) and forward them to the real implementation.
- `proc.c` or `vm.c` contains the actual implementation.

Listing 3: Trap-number-to-handler dispatch in `syscall.c`

```
1 static int (*syscalls[])(void) = {
2     [SYS_fork]      sys_fork,
3     ...
4     [SYS_worldpeace] sys_worldpeace,
5     [SYS_numberofprocesses] sys_numberofprocesses,
6     [SYS_spawn]     sys_spawn,
7     ...
8 };
9
10 void syscall(void) {
11     int num = myproc()->tf->eax;
12     if (num > 0 && num < NELEM(syscalls) && syscalls[num])
13         curproc->tf->eax = syscalls[num]();
14     else { ... curproc->tf->eax = -1; }
15 }
```

This same pipeline is reused for every call below, so the descriptions that follow focus on the kernel-side logic rather than repeating the wiring.

(a) `worldpeace()`

The simplest call in the lab: it takes no arguments and prints a fixed message from kernel mode.

Listing 4: `sys_worldpeace` in `sysproc.c`

```
1 int sys_worldpeace(void) {
2     cprintf("Systems are vital to world peace !!\n");
3     return 0;
4 }
```

The reason this uses `cprintf` rather than the ordinary `printf` that user programs use is that `printf` is a *user-space* library routine (in `ulib.c`) that itself calls the `write` system call; it only exists in the context of a running user process writing to a file descriptor. Kernel code has no file descriptors and no concept of “process 1’s console” separate from the physical console device — `cprintf` writes directly to the console driver, protected by its own lock so concurrent kernel messages from different CPUs interleave a whole message at a time rather than character by character.

(b) `numberofprocesses()`

This call answers “how many processes are currently READY (RUNNABLE in xv6’s terminology, i.e. eligible to run but not currently scheduled)?” by walking the kernel’s global process table.

Listing 5: `numberofprocesses` in `proc.c`

```
1 int numberofprocesses(void) {
2     struct proc *p;
3     acquire(&ptable.lock);
4     int ready_count = 0;
5     for (p = ptable.proc; p < &ptable.proc[NPROC]; p++)
6         if (p->state == RUNNABLE)
7             ready_count++;
8     release(&ptable.lock);
9     return ready_count;
10 }
```

xv6 has no dynamic process list; `ptable.proc` is a fixed-size array of `NPROC` process control blocks (PCBs), most of which sit in the `UNUSED` state at any given time. Every function that needs process-table-wide information — this one, `spawn`, `wait`, the scheduler itself — follows the identical pattern of taking `ptable.lock`, doing a linear scan of the whole array, and releasing the lock. The lock exists because the table is shared mutable state across CPUs: without it, a process’s state field could be read mid-transition (say, between being marked `RUNNABLE` by a wakeup and picked up by the scheduler on another core) and produce a torn or stale count.

(c) `spawn(int n, int* pids)`

`spawn` creates `n` children in one call, returns 0 to each child and the number of children created to the parent, and fills the caller-supplied `pids` array with the children’s process IDs so the parent can reap them afterward.

The natural first instinct is to loop calling `fork()` `n` times, but that does not work directly: `fork()` returns 0 in the child, and a loop body that receives 0 has no way to distinguish “I am a freshly spawned child, stop looping” from “continue spawning more children” without extra bookkeeping, and doing so from inside the loop risks the child re-entering the `spawn` logic itself. The submitted implementation avoids this altogether by not calling the existing `fork()` wrapper at all, and instead inlining the process-creation steps that `fork()` itself performs, directly in a loop that only ever executes in the parent:

Listing 6: `spawn` in `proc.c` (abridged)

```
1 int spawn(int n, int *pids) {
2     int count = 0;
3     for (int i = 0; i < n; i++) {
4         struct proc *curproc = myproc();
5         struct proc *np;
```

```

6     if ((np = allocproc()) == 0) return -1;
7     count++;
8     if ((np->pgdir = copyuvm(curproc->pgdir, curproc->sz)) == 0) { ... return -1;
9         }
10    np->sz = curproc->sz;
11    np->parent = curproc;
12    *np->tf = *curproc->tf;
13    np->tf->eax = 0;           // child's spawn() returns 0
14    for (int j = 0; j < NOFILE; j++)
15        if (curproc->ofile[j]) np->ofile[j] = filedup(curproc->ofile[j]);
16    np->cwd = idup(curproc->cwd);
17    pids[i] = np->pid;
18    acquire(&ptable.lock);
19    np->state = RUNNABLE;
20    release(&ptable.lock);
21 }
22 return count;
23 }

```

Each iteration performs the same four steps `fork()` does: `allocproc()` claims a free PCB slot and sets up a kernel stack for it; `copyuvm()` allocates a fresh physical address space and copies the parent's memory contents into it page by page; the new process's saved trap frame is initialized as a byte-for-byte copy of the parent's, with only `eax` overwritten to 0 so that when this new process is eventually scheduled, the same `spawn()` call it "was in the middle of" appears to have already returned with a value of 0; and the new PCB is marked `RUNNABLE`. Because none of this touches the calling process's own control flow the way `fork()`'s dual return does, the parent can keep looping and keep populating `pids[]` without the new children ever executing `spawn`'s own loop body — a child only starts running from the point captured in its copied trap frame, i.e. from just after the original `int $T_SYSCALL` instruction in `usys.S`, with `eax = 0` already in place. The parent's subsequent `wait()` calls (three of them, matching the three children, per the test program) then reap the children in whichever order they exit.

Task 3: Virtual and Physical Addresses

(a) `getvasize(int pid)`

Returns the size, in bytes, of a process's virtual address space by looking up its PCB and reading `proc->sz` directly:

Listing 7: `getvasize` in `proc.c`

```

1 int getvasize(int pid) {
2     struct proc *currproc;
3     uint size = -1;
4     acquire(&ptable.lock);
5     for (currproc = ptable.proc; currproc < &ptable.proc[NPROC]; currproc++)
6         if (currproc->pid == pid) { size = currproc->sz; break; }
7     release(&ptable.lock);
8     return size;
9 }

```

`proc->sz` is not incidental bookkeeping; it *is* xv6's definition of a process's virtual address space size, and every other memory-management primitive treats it as authoritative. `sbrk(n)`, implemented

in `growproc`, works by calling `allocvm` to extend the page table from the current `sz` to `sz + n` and then storing the new size back into `proc->sz`; this lab’s `mmap` does the same, just without the `allocvm` step (Section 5). Address space in `xv6` always starts at virtual address 0 and grows upward, with everything below `sz` nominally valid and everything at or above it nominally not — that boundary is exactly the check the page fault handler uses to decide whether a fault is a lazy-allocation opportunity or an illegal access (Section 5).

(b) `va2pa(uint virtual_addr)`

Implemented as `va_to_pa`, this call resolves a virtual address to its physical address by walking the *current* process’s page table:

Listing 8: `va_to_pa` in `vm.c`

```

1 uint va_to_pa(uint virtual_addr) {
2     pte_t *pte = walkpgdir(myproc()->pgdir, (const void *)virtual_addr, 0);
3     if (*pte != 0 && *pte & PTE_P) {
4         uint first_20 = PTE_ADDR(*pte);
5         uint last_12  = PTE_FLAGS(virtual_addr); // page offset bits
6         return first_20 | last_12;
7     }
8     return -1;
9 }

```

`x86`’s two-level paging splits a 32-bit virtual address into a 10-bit page directory index, a 10-bit page table index, and a 12-bit page offset. `walkpgdir(pgdir, va, 0)` does exactly what the paging hardware would do to resolve that address, entirely in software: it indexes the page directory with the top 10 bits, follows the present page-directory entry to the second-level page table, and returns a pointer to the page table entry (PTE) selected by the next 10 bits (the trailing 0 argument means “do not allocate a new page table page if one is missing here” — this call is read-only). The physical address is then the PTE’s stored 20-bit physical page number combined with the original 12-bit offset from the virtual address, since paging translates only the page number, never the within-page offset.

The lab’s `t_va_to_pa2.c` test deliberately surfaces something the handout calls out as “interesting”: it forks, and both parent and child print the virtual and the translated physical address of the *same stack variable* `N`. The virtual address printed is identical in both processes — unsurprising, since the child’s stack layout is a byte-for-byte copy of the parent’s at the moment of `fork` — but the physical address differs between them. This is a direct, visible consequence of how `fork()` is implemented (Section 3.3): `copyvm` allocates an entirely *new* set of physical pages for the child and copies the parent’s page contents into them; `xv6` has no copy-on-write. So while the child’s page table maps virtual address `va` to a new physical frame containing a copy of `N`’s value, the parent’s page table still maps that same `va` to its own original physical frame. Two processes, same virtual address, genuinely different physical memory — which is exactly the isolation guarantee virtual memory is supposed to provide, made directly observable here through `va2pa`.

(c) Page table size: `get_pgtb_size`, `get_usr_pgtb_size`, `get_kernel_pgtb_size`

These three calls report how many second-level page table pages are currently allocated for the calling process’s address space, split into totals, the user half, and the kernel half.

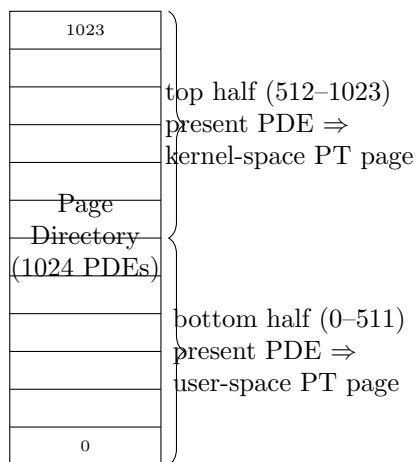
Listing 9: Page table page counting in `vm.c`

```

1 int get_pgtb_size(void) {
2     pde_t *pgdir = myproc()->pgdir;
3     int count = 0;
4     for (uint i = 0; i < NPTENTRIES; i++)
5         if (pgdir[i] & PTE_P) count++;
6     return count;
7 }
8 // get_usr_pgtb_size: i in [0, NPTENTRIES/2)
9 // get_kernel_pgtb_size: i in [NPTENTRIES/2, NPTENTRIES)

```

xv6’s page table is a two-level tree: a single page directory of 1024 entries, each of which (if present) points to a second-level page table of 1024 PTEs. Because a 32-bit address space is split so that virtual addresses below `KERNBASE` are user space and everything from `KERNBASE` upward is kernel space, and `KERNBASE` sits exactly at the halfway point of the address space, the top half of the page *directory* (indices 512–1023) corresponds entirely to kernel mappings and the bottom half (0–511) to user mappings. Counting present page-directory entries is therefore exactly counting how many second-level page table *pages* have been allocated — one per present PDE — which is what “size of the page table” means in this call’s context (as opposed to counting individual PTEs, which would measure mapped memory rather than page-table *structure*). Splitting the same loop into the first and second halves of the 1024-entry directory is what separates the user-space count from the kernel-space count. This matches the lab’s own worked example: a fresh process shows one user-space page table page (for its small text/data/stack footprint) and a fixed 64 kernel-space page table pages (the kernel mappings xv6 installs identically into every process’s page table so that trap handling can run without switching page tables), for a total of 65.



`get_kernel_pgtb_size`: count present PDEs in top half
`get_usr_pgtb_size`: count present PDEs in bottom half
`get_pgtb_size`: count present PDEs over all 1024

(d) `getpasize(int pid)`

Where the previous calls count page table *structure* (how many PTE pages exist), `getpasize` counts actual mapped physical *memory* — the number of user-space pages that currently have a valid physical frame behind them.

Listing 10: `getpasize_of_pgdir` in `vm.c`

```

1 int getpasize_of_pgdir(pde_t *pgdir) {

```

```

2  int count = 0;
3  for (uint i = 0; i < NPENTRIES / 2; i++) {           // user-space PDEs only
4      if (pgdir[i] & PTE_P) {
5          pte_t *pg_2 = (pte_t *)P2V(PTE_ADDR(pgdir[i]));
6          for (uint j = 0; j < NPENTRIES; j++)
7              if (pg_2[j] & PTE_P) count++;           // present PTE = mapped page
8      }
9  }
10 return count;
11 }

```

This is a genuine two-level walk rather than a single loop, because a present page-directory entry only guarantees that a *second-level page table* exists, not that every (or any) slot in that page table maps a real page — a PDE can be present while most of its PTEs are still zero. The outer loop restricts itself to the user half of the directory deliberately: “physical memory allocated to *the process*” is a statement about that process’s own data, not about the kernel mappings every process’s page table also carries. The function is invoked with a process’s `pgdir` looked up from its PCB the same way `getvasize` finds `sz`, which is why the two calls together let the test program contrast virtual size against actually-backed physical size — the gap between them is exactly the lazily-unmapped region exercised in Section 5.

Task 4: Page Faults, mmap, and Lazy Allocation

(a) Explicit page fault reporting

The default xv6 trap handler has no dedicated case for page faults; an unhandled trap falls through to the generic “kill the offending process” path in `trap()`’s `default` branch, which does print the faulting address (via `rcr2()`, the x86 register the CPU populates with the faulting virtual address on a page fault) but only as part of a generic message shared with every other unrecognized trap. The lab’s first sub-task adds a dedicated case `T_PGFLT` to `trap()`’s `switch` so page faults are recognized and reported explicitly rather than falling into that generic path. In the submitted kernel this dedicated case was written directly against the final requirement of the task rather than as an intermediate step: it already distinguishes a fault that should be serviced lazily from one that is genuinely illegal (Section 5.3), and only emits the diagnostic `cprintf` — reporting the faulting instruction pointer and address — on the illegal-access path, since a successfully-serviced lazy fault is not an error condition worth logging.

(b) `mmap(int nbytes)`

`mmap` grows a process’s virtual address space, exactly like `sbrk`, but — unlike `sbrk`, which calls `allocuvn` immediately to back the new range with real physical pages and PTEs — `mmap` only moves the boundary of what counts as valid virtual address space, and defers allocating any physical memory or page table entries until the new region is actually touched.

Listing 11: `sys_mmap` (`sysproc.c`) and `grow` (`proc.c`)

```

1  int sys_mmap(void) {
2      int size;
3      if (argint(0, &size) < 0) return 0;
4      int addr = myproc()->sz;
5      if (grow(size) < 0) return 0;
6      return addr;
7  }

```



```

8
9 int grow(int n) {
10     if (n <= 0 || n % PGSIZE != 0) return -1;
11     myproc()->sz += n;
12     return 0;
13 }

```

`grow` validates the two conditions the spec requires — a positive byte count that is a whole multiple of `PGSIZE` — and, if valid, simply increments `sz`. No call to `allocvm`, no new PTEs. The returned value, the process’s size *before* the call, is the starting virtual address of the newly added region, mirroring how `sbrk` reports the old break as the address of freshly available space. At this point the process’s virtual address space is larger, but its page table and physical memory footprint are completely unchanged: a valid virtual address now exists in a range that resolves to nothing when walked, which is precisely the situation the page fault handler in Section 5.3 is built to catch.

(c) Lazy allocation on page fault

The final piece ties the previous two together: rather than pre-mapping memory at `mmap` time, physical pages are allocated only when the process actually dereferences an address inside the `mmap`’d region, at which point the resulting page fault is caught, serviced by mapping in a real page, and the faulting instruction is transparently retried.

Listing 12: T_PGFLT case in `trap()`

```

1 case T_PGFLT:
2 {
3     uint addr = rcr2();
4     uint pground_addr = PGROUNDDOWN(addr);
5     struct proc *curproc = myproc();
6     if (addr < curproc->sz) {
7         allocvm(curproc->pgdir, pground_addr, pground_addr + PGSIZE);
8         switchvm(curproc);
9     } else {
10         cprintf("Pagefault occured at address eip 0x%x addr 0x%x--kill proc\n",
11                 tf->eip, addr);
12         myproc()->killed = 1;
13     }
14     break;
15 }

```

Several details here are load-bearing rather than incidental. `rcr2()` reads the CR2 control register, which the x86 paging hardware populates with the faulting linear address at the moment it raises a page fault — this is how the handler learns *which* address triggered the trap, since that information is not otherwise passed on the stack the way a syscall number is. `PGROUNDDOWN(addr)` rounds the faulting address down to the start of its containing 4KB page, because `allocvm` maps whole pages: a fault partway through a page must still result in the *entire* page being mapped, not just the faulting byte. The validity check `addr < curproc->sz` is what distinguishes a “good” fault — one on an address inside the process’s declared virtual size that simply has not been backed by physical memory yet, exactly the state `mmap` leaves behind — from a genuinely illegal access on an address the process has no business touching (a wild pointer, a null dereference past the mapped region, etc.), which is still killed exactly as the unmodified handler would.

The `switchvm(curproc)` call after `allocvm` is not optional bookkeeping. x86 caches virtual-

to-physical translations in the TLB, and more fundamentally the CPU's CR3 register holds the physical address of the page directory currently in use; simply editing `curproc->pgdir` in memory does not make the hardware re-read it. `switchvm` reloads CR3 (and, as a side effect of the reload, flushes stale TLB entries for this address space), so that when `trap()` returns and the process resumes at the exact instruction that faulted, the hardware's page-table walk for that address now finds the freshly-installed, present PTE instead of faulting again. Without this call the same instruction would fault indefinitely: the mapping would exist in the in-memory page table but never become visible to the CPU actually executing the process. This is also why the handler does not need to manually adjust the trap frame's instruction pointer to "retry" — a page fault trap on x86 already leaves `eip` pointing at the faulting instruction itself (unlike, say, a syscall trap, whose return address is the instruction *after* the trapping one), so simply returning from the handler after fixing the mapping causes the CPU to naturally re-execute that instruction, which now succeeds.

The combined test program `t_lazy.c` demonstrates this end to end: it calls `mmap(3 * PGSIZE)`, confirms via `getvsize` that virtual size grew immediately by three pages while `getpasive` shows physical size unchanged, then touches each of the three pages in turn and re-checks both counts — observing physical size climb by exactly one page per access, confirming allocation is happening lazily and precisely on demand rather than all at once.

Optional Exercises

Three optional exercises were listed in the handout. Two were implemented and tested; the third (custom trap-number system calls) was not attempted, and no corresponding trap-65 handler, `fork2` entry point, or `userfork2` test binary exists in the submitted source or `Makefile` program lists.

`get_sibling()`

Prints and returns the count of processes sharing the calling process's parent:

Listing 13: `get_sibling` in `proc.c`

```

1 int get_sibling(void) {
2     struct proc *currproc;
3     uint parent_id = myproc()->parent->pid;
4     acquire(&ptable.lock);
5     for (currproc = ptable.proc; currproc < &ptable.proc[NPROC]; currproc++)
6         if (currproc->parent->pid == parent_id) {
7             cprintf("%d %s\n", currproc->pid, get_status_string(currproc));
8         }
9     release(&ptable.lock);
10    return 0;
11 }
```

This reuses the same `ptable`-scan-under-lock idiom as `numberofprocesses`, but compares each candidate's `parent` pointer against the caller's own parent rather than inspecting the candidate's own state directly — "siblings" is a relationship defined by a shared parent, including the calling process itself, since it also matches its own parent's pid. The corresponding test program, `mysiblings.c`, and its `Makefile` entry (`_mysiblings`) are present and unmodified from the provided harness, confirming this was exercised.

`whatsthestatus(int pid)`

Given a pid, reports (and prints) that process's parent pid, parent name, and current scheduling state:

Listing 14: `whatsthestatus` in `proc.c`

```
1 int whatsthestatus(int pid) {
2     struct proc *currproc = myproc();
3     acquire(&ptable.lock);
4     for (currproc = ptable.proc; currproc < &ptable.proc[NPROC]; currproc++)
5         if (currproc->pid == pid) {
6             cprintf("%d %s %d %s\n", pid, get_status_string(currproc),
7                     currproc->parent->pid, currproc->parent->name);
8         }
9     release(&ptable.lock);
10    return currproc->parent->pid;
11 }
```

Both this call and `get_sibling` share a small helper, `get_status_string`, that maps the internal `enum procstate` values (`UNUSED`, `EMBRYO`, `SLEEPING`, `RUNNABLE`, `RUNNING`, `ZOMBIE`) to their printable names — avoiding duplicating that switch statement across both calls. The `_status` binary is present in the Makefile's custom program list, matching the handout's `status.c` test harness.

Build System: From Host Source to Booted Kernel

Because xv6 cannot compile or edit its own source, every task in this lab required a matching Makefile change in addition to the C code, and it is worth being explicit about what each change does, since a program or data file that exists in the source tree but is missing from these lists simply never appears inside the running OS.

Listing 15: Relevant Makefile lists (abridged)

```
1 CUSTOMPROGS1=\
2     _hw\ _mycat\ _pingpong\ _cmd\ _worldpeace\ _nump\ _spawn\
3
4 CUSTOMPROGS2=\
5     _t_getvasize\ _t_va_to_pa1\ _t_va_to_pa2\ _t_getpgtables\
6     _t_getpaside\ _t_pagefault\ _t_mmap\ _t_lazy\
7     _mysiblings\ _status\
8
9 EXTRA_FILES=\
10    abc.txt\ pingpong.txt\
11
12 fs.img: mkfs README $(UPROGS) $(CUSTOMPROGS1) $(CUSTOMPROGS2) $(EXTRA_FILES)
13    ./mkfs fs.img README $(UPROGS) $(CUSTOMPROGS1) $(CUSTOMPROGS2) $(EXTRA_FILES)
```

Each name prefixed with an underscore (e.g. `_pingpong`) is a Makefile pattern target that compiles and links the corresponding `.c` file into a standalone xv6 ELF binary using the cross-compiler toolchain, entirely on the host — xv6 never runs a compiler itself. The `fs.img` rule then invokes `mkfs`, a small host-side program that constructs a raw xv6 filesystem image and writes each listed binary and data file into it as a regular file. `mkfs` is what makes `pingpong.txt` available to read from inside the running OS: it is not a source file at all from xv6's point of view, just a data file copied byte-for-byte into the disk image alongside the compiled programs, listed in `EXTRA_FILES`.

rather than `CUSTOMPROGS` because `mkfs` treats it as an entry to embed, not a target to compile. `xv6.img` (built separately from `bootblock` and `kernel`) and `fs.img` are then handed to QEMU as two virtual disks — the first boots the kernel, the second is mounted as the root filesystem the shell and every test program above run from.

Key Design Decisions and Tradeoffs

A few choices made while implementing this lab are not forced by the spec and are worth calling out explicitly, since each involves a real tradeoff rather than a single obviously-correct answer.

Inlining process creation in `spawn` instead of calling `fork()`. As discussed in Section 3.3, looping over `fork()` does not straightforwardly produce the semantics `spawn` needs, because `fork()`'s dual return makes the loop body itself ambiguous between parent and child control flow. The chosen alternative — duplicating `fork()`'s four steps (`allocproc`, `copyvm`, trap frame copy, file descriptor/cwd duplication) directly inside `spawn` — avoids that ambiguity at the cost of code duplication between `fork()` in `proc.c` and `spawn()` right below it. The submitted source keeps the `fork()`-based version as a commented-out alternative directly above the working implementation, which is a reasonable way to document that the simpler approach was tried first and rejected for a concrete reason, rather than never considered.

No copy-on-write in `fork()/spawn()`. Every child process — whether created by `fork` or by `spawn` — gets a full physical copy of the parent's address space via `copyvm`, made visible in Section 4.2's parent/child `va2pa` divergence. This is `xv6`'s existing design, inherited rather than introduced by this lab, but it directly shapes what `spawn(n, ...)` costs: creating `n` children means `n` full page-by-page copies of the calling process's memory, proportional to process size and independent of how much of that memory the children actually go on to touch. A copy-on-write scheme would defer (and often avoid) that copying entirely, at the cost of the extra page-fault bookkeeping this lab's own lazy-allocation task (Section 5.3) shows is non-trivial to get right — which is precisely why `xv6`, as a teaching kernel, keeps the simpler, more wasteful full-copy semantics instead.

One trap-handler case serving two spec requirements. Section 5.1's explicit-page-fault-reporting requirement and Section 5.3's lazy-allocation requirement are written as sequential sub-tasks in the handout, and could have been implemented as two independent code paths — an early case that always logs pid and faulting address, followed by separate logic that additionally services lazy faults. The submitted `trap.c` instead merges them into a single `case T_PGFLT` that branches once, at the top, on whether the fault is serviceable (`addr < curproc->sz`) and only logs and kills on the branch that is not. This means an `mmap`'d-but-unbacked page produces no diagnostic output at all when serviced — which matches the handout's own hint to "return in order to avoid the `cprintf`" once a fault is correctly handled — at the cost of the reporting behavior only being observable for faults the kernel treats as illegal, not for every fault `T_PGFLT` ever sees.

Limitations and Known Gaps

- **Chunked substring matching in `pingpong`.** Because the file is read in fixed 512-byte chunks and matching is done purely within each in-memory buffer, an occurrence of `ping` that straddles a chunk boundary (e.g. `...pi` at the end of one `read()` and `ng...` at the start of the next) would not be detected. The provided `pingpong.txt` never triggers this, so

it was not a practical issue, but it is a real edge case the current implementation does not handle.

- **Unvalidated failure path in spawn.** If `allocproc` or `copyvm` fails partway through spawning `n` children, the function returns `-1` immediately, but any children already created in earlier loop iterations are left `RUNNABLE` in the process table with their pids already written into `pids[]` — the caller has no signal telling it how many, if any, children to reap in this failure case.
- **`mmap` does not validate against the reachable address ceiling, and the lazy-fault path does not check `allocvm`'s return value.** `grow()` only checks that the requested size is positive and page-aligned; it does not check whether growing `sz` by that amount would ever be satisfiable. If a process later touches a page in a region that pushes past `KERNBASE` or that exhausts physical memory, `allocvm` (which does check `newsz >= KERNBASE` and can fail on out-of-memory) returns 0, but the `T_PGFLT` case in `trap.c` discards that return value and calls `switchvm` unconditionally — so the still-unmapped access would fault again on retry rather than being detected and the process killed cleanly.
- **The third optional exercise (custom trap-number system calls / fork2 via trap 65) was not attempted.** No trap-65 IDT entry, no `fork2` `usys.S` trampoline, and no `userfork2/_userfork2` Makefile target exist in the submitted tree.

Testing and Results

Every task in this lab shipped with a matching user-space test program — `worldpeace.c`, `nump.c`, `spawn.c`, `t_getvasize.c`, `t_va_to_pa1/2.c`, `t_getpgtablesz.c`, `t_getpaside.c`, `t_pagefault.c`, `t_mmap.c`, and `t_lazy.c`, plus `mysiblings.c` and `status.c` for the optional exercises — and all of them are wired into the `CUSTOMPROGS1/CUSTOMPROGS2` lists described above, so they are compiled into `fs.img` on every build alongside `pingpong.c` and `cmd.c` from Task 1. Running each test binary under QEMU exercises the corresponding kernel path end to end: system call dispatch through the `syscalls[]` table, ptable traversal for process-info calls, page-table walks for the address-translation calls, and the extended trap handler for the fault-driven calls.

The two page-table-walking calls in Section 4 — `va_to_pa` and the `get.*pgtb_size/getpaside` family — and the lazy allocation path in Section 5 are the parts of this lab that most directly exercise xv6's memory management internals rather than its process management: they required reading and reusing `walkpgdir`, `allocvm`, and `switchvm` rather than writing memory-management logic from scratch, which mirrors how xv6 itself is built — `sbrk`, `exec`, and `fork` all share the same small set of address-space primitives in `vm.c` rather than each reimplementing page-table manipulation independently. The lazy-allocation task in particular is a direct, minimal illustration of demand paging, the same mechanism (in more elaborate form) that lets a real OS overcommit memory and back large virtual regions — memory-mapped files, sparse heaps — without physically allocating pages that are never touched.